STANDAR OPERASIONAL PROSEDUR (SOP)

Alur Kode Lokal hingga Push ke GitLab UMY

https://student-git.umy.ac.id/

Berlaku untuk Windows, macOS, Linux, dan WSL

1. Ruang Lingkup

- SOP ini digunakan ketika pengembang memulai pekerjaan dari repository lokal, melakukan perubahan kode, validasi hasil, commit, dan push ke GitLab.
- Prosedur ini berlaku untuk semua anggota tim yang menggunakan Git sebagai version control dan GitLab UMY (student-git.umy.ac.id) sebagai remote repository.

2. Prasyarat

- Git telah terinstal pada perangkat kerja.
- Akun GitLab UMY (student-git.umy.ac.id) sudah aktif dan memiliki akses ke repository tujuan. Gunakan akun kampus (@mail.umy.ac.id)
- Pengembang memiliki kredensial SSH atau HTTPS token sesuai kebijakan repository.
- Code editor, misalnya VS Code, JetBrains, atau editor lain yang setara.
- Perangkat terhubung ke jaringan kampus, GitLab (student-git.umy.ac.id) hanya dapat diakses dari jaringan UMY.

3. Standar Kerja Umum

- Seluruh perubahan wajib dikerjakan di branch kerja, bukan langsung di branch utama.
- Sebelum commit, pastikan kode sudah diuji secara lokal dan tidak ada file konfigurasi sensitif yang ikut ter-track.
- Gunakan pesan commit yang deskriptif dan konsisten.
- Sebelum push, sinkronkan branch lokal dengan remote untuk mengurangi konflik merge.

4. Prosedur Operasional

4.1 Inisialisasi repository lokal

- Buka terminal pada folder proyek.
- Jika repository belum ada, lakukan clone dari GitLab UMY (student-git.umy.ac.id) atau init repository baru sesuai kebutuhan.

```
git clone git@student-git.umy.ac.id:kelompok/nama-project.git
# atau via HTTPS
git clone https://student-git.umy.ac.id/kelompok/nama-project.git
cd project
```

4.2 Membuat branch kerja

- Buat branch baru untuk setiap fitur, perbaikan bug, atau perubahan konfigurasi.
- Nama branch sebaiknya menggambarkan tujuan perubahan.

```
git checkout -b feature/nama-fitur
# atau
git switch -c feature/nama-fitur
```

4.3 Melakukan perubahan dan validasi lokal

- Implementasikan perubahan di codebase.
- Lakukan pengujian lokal, build, atau running aplikasi sesuai stack yang digunakan.
- Pastikan tidak ada error runtime, path salah, atau file sensitif yang tidak semestinya ikut berubah.

4.4 Memeriksa status perubahan

- Gunakan status Git untuk melihat file yang berubah.
- Pastikan hanya file relevan yang akan di-commit.

```
git status
git diff
```

4.5 Staging file

- Tambahkan file yang memang perlu masuk ke commit.
- Hindari menambahkan file environment, credential, log, build artifact, atau file sementara.

```
git add <file>
git add .
```

4.6 Commit perubahan

- Buat commit dengan pesan yang ringkas, spesifik, dan mudah dilacak.
- Format commit dapat menggunakan pola: type(scope): message.

```
git commit -m "feat(auth): add login validation"
```

4.7 Sinkronisasi dengan remote

- Jika branch remote sudah diperbarui, lakukan pull/rebase agar branch lokal tetap sinkron.
- Selesaikan konflik merge bila ada sebelum push.

```
git pull --rebase origin feature/nama-fitur
```

4.8 Push ke GitLab UMY

- Push branch lokal ke GitLab setelah commit berhasil dan pengujian lokal lulus.
- Verifikasi bahwa branch dan commit sudah muncul di repository remote.

```
git push -u origin feature/nama-fitur
```

4.9 Review dan merge request

- Buat Merge Request (MR) jika workflow proyek mewajibkan review.
- Pastikan deskripsi MR berisi ringkasan perubahan, alasan, dan dampak teknis.

5. Panduan Per Platform

Windows

- Gunakan Git Bash, PowerShell, atau terminal dari VS Code.
- Jika memakai Git Bash, perintah pada SOP dapat dijalankan langsung tanpa penyesuaian berarti.
- Pastikan line ending konsisten dengan konfigurasi repository.

macOS

- Gunakan Terminal bawaan, iTerm2, atau terminal lain yang mendukung Git.
- Jika repository disimpan pada disk eksternal atau folder sinkronisasi, pastikan permission file tidak menghambat operasi Git.

Linux

- Gunakan terminal distribusi yang dipakai, misalnya bash, zsh, atau fish.
- Pastikan permission folder proyek tidak menghalangi operasi write saat commit dan push.

WSL

- Simpan repository di filesystem Linux WSL, misalnya di /home/<user>/project, untuk performa yang lebih stabil.
- Hindari workflow aktif di /mnt/c jika repository sering melakukan operasi file besar atau banyak file kecil.
- Gunakan Git yang terpasang di WSL untuk mencegah perbedaan permission dan line ending.

6. Aturan Teknis

Aspek	Standar
Branch	main/master hanya untuk rilis stabil; perubahan development memakai branch terpisah
Commit message	Ringkas, deskriptif, konsisten, dan mudah ditelusuri
Secret management	Tidak boleh hardcoded; simpan di .env atau secret manager
File sensitif	Tidak di-commit ke repository
Validasi lokal	Wajib dilakukan sebelum push

7. Output yang Diharapkan

- Repository lokal dan remote memiliki riwayat commit yang bersih dan terstruktur.
- Perubahan dapat ditelusuri berdasarkan branch dan pesan commit.
- Kode yang dipush ke GitLab sudah melalui validasi lokal.
- Tidak ada credential, konfigurasi sensitif, atau file sementara yang ikut terpublikasi.

8. Catatan Operasional

- Jika menggunakan GitLab CI/CD, pastikan pipeline berjalan setelah push untuk memverifikasi integritas build.
- Jika terjadi konflik, lakukan resolve di branch kerja dan ulangi validasi lokal sebelum push ulang.
- Dokumen ini dapat disesuaikan dengan kebijakan tim, tetapi struktur minimal alur kerja harus tetap dipertahankan.

Activity Diagram GitLab UMY

